

Sistema Web Integrado à Inteligência Artificial Generativa para o Gerenciamento do Treinamento de Hipertrofia em Praticantes Experientes

João Vitor dos Santos Ritter, Fabiano Niederauer Flôres

Curso de Sistemas de Informação

UFN - Universidade Franciscana

Santa Maria - RS

joao.ritter@ufn.edu.br, fabiano.flores@ufn.edu.br

Resumo—O treinamento de força voltado à hipertrofia muscular exige, em praticantes experientes, um controle rigoroso de variáveis como volume semanal, Percepção Subjetiva de Esforço (RPE) e Repetições em Reserva (RIR), cujo cruzamento analítico é de difícil sustentação manual a longo prazo. Diante dessa complexidade, este trabalho propõe o desenvolvimento de um sistema web integrado com Inteligência Artificial (IA) Generativa, utilizando a análise ativa de variáveis de esforço para auxiliar praticantes experientes na otimização de seus resultados e progressão de carga. A solução é estruturada sob o padrão arquitetural MVC (*Model-View-Controller*), utilizando React, Node.js e PostgreSQL, e integrada à API (*Application Programming Interface*) do Google Gemini por meio de técnicas de Engenharia de Prompts. O desenvolvimento é conduzido pela metodologia ágil FDD (*Feature-Driven Development*), orientando a construção incremental da aplicação. Nesta primeira etapa, foram produzidos os diagramas de modelagem do sistema, o levantamento de requisitos funcionais e não funcionais, o *backlog* de funcionalidades e a especificação da anatomia do *prompt* estruturado. A etapa seguinte prevê a implementação completa do protótipo executável e sua validação por meio de testes técnicos de unidade e integração, avaliação heurística de usabilidade e verificação de coerência dos diagnósticos gerados pela IA com a literatura científica.

Palavras-chave : Treinamento de Força; Hipertrofia Muscular; Inteligência Artificial; Engenharia de Prompts; Aplicação Web.

I. INTRODUÇÃO

O treinamento de força voltado à hipertrofia muscular evoluiu de uma prática empírica para uma disciplina fundamentada em dados e evidências científicas. Com o avanço das tecnologias de informação, surgiram diversas soluções de software para auxiliar praticantes na organização de suas rotinas de treino e o registro histórico de atividades físicas [1].

Entretanto, a utilidade dessas ferramentas depende do nível de experiência do usuário. Em praticantes iniciantes, treinos simples já podem gerar resultados significativos, visto que o corpo ainda responde com facilidade aos novos estímulos. Em praticantes experientes, esse processo se torna mais difícil, uma vez que o organismo já está adaptado ao treinamento de força. Dessa forma, para continuar evoluindo, esse público precisa de um planejamento mais criterioso,

com controle de variáveis como volume semanal, intensidade do esforço e recuperação [2, 3].

Essa necessidade de controle exato das variáveis de esforço é justificada pelas adaptações neuromusculares do praticante experiente; de acordo com Zourdos et al. [4], atletas experientes possuem uma eficiência motora superior para recrutar unidades motoras de alto limiar e suportar a verdadeira exaustão, diferentemente de iniciantes que frequentemente não conseguem atingir a falha real com cargas máximas. Logo, para que ocorra a contínua sinalização de hipertrofia nessa categoria de praticante experiente, o estímulo prescrito precisa ser monitorado e ajustado de forma muito mais rigorosa. O que exige um controle muito mais criterioso das variáveis de esforço para que ocorra a quebra da homeostase e a contínua sinalização de hipertrofia.

Nesse estágio avançado, a simples execução dos exercícios é insuficiente, o controle preciso das variáveis de esforço é o que define a continuidade da evolução ou a estagnação em um platô. Torna-se indispensável o monitoramento do volume semanal de séries por grupamento muscular, uma métrica que possui forte relação dose-resposta com o aumento da massa muscular, conforme evidenciado por Schoenfeld, Ogborn e Krieger [5]. Paralelamente, exige-se o controle da intensidade relativa, frequentemente mensurada pela RPE (*Rating of Perceived Exertion*, que significa Percepção Subjetiva de Esforço), e pelas RIR (*Repetitions in Reserve*, traduzida para o português como Repetições em Reserva), métodos validados por Zourdos et al. [4] e Helms et al. [6] para quantificar proximidade da falha muscular. O grande desafio, portanto, reside na capacidade de interpretar e cruzar esses dados técnicos simultaneamente para garantir que o estímulo aplicado seja, de fato, sinalizador de hipertrofia.

Considerando essa complexidade analítica, a Engenharia de Software aliada à Inteligência Artificial (IA) apresenta-se como uma abordagem viável para auxiliar neste processo. A estruturação de sistemas web baseados em padrões de arquitetura, como o MVC (*Model-View-Controller*), e desenvolvidos por meio de um ecossistema computacional, incluindo a biblioteca React para interfaces de usuário, o ambiente Node.js para o processamento do lado do servidor

e o sistema PostgreSQL para a persistência de dados, viabiliza a coleta e o armazenamento estruturado destas métricas.

Buscando diferenciar-se dos sistemas focados apenas no registro manual, propõe-se a integração dessas arquiteturas com APIs (*Application Programming Interfaces*) de IA Generativa. Através da aplicação de técnicas de engenharia de prompts, torna-se possível utilizar o modelo para o processamento em conjunto das variáveis (Volume, RIR e RPE), transformando dados brutos em ferramentas de suporte à decisão que permitem fornecer diagnósticos precisos e automatizados para a rotina do usuário.

A. Objetivo Geral

Desenvolver um sistema web integrado com IA voltado ao gerenciamento da hipertrofia muscular, utilizando a análise ativa de variáveis de esforço para auxiliar praticantes experientes na otimização de seus resultados e progressão de carga.

B. Objetivos Específicos

- Identificar e mapear as principais métricas de treinamento (volume semanal, RPE e RIR) fundamentadas na literatura científica para a sinalização de hipertrofia;
- Desenvolver a arquitetura do sistema no padrão MVC, construindo a interface de usuário em React, o processamento de regras de negócio em Node.js e a persistência de dados em PostgreSQL;
- Integrar a aplicação à API de Inteligência Artificial Generativa (Google Gemini) utilizando técnicas de engenharia de prompts para processar métricas brutas e retornar diagnósticos de treino ao usuário;
- Validar o protótipo por meio de testes de software e simulações computacionais de cenários de treino, verificando a eficácia do diagnóstico gerado pela IA e a usabilidade da interface através de avaliação heurística.

II. REFERENCIAL TEÓRICO

Nesta seção, apresenta-se a fundamentação teórica relacionada à proposta do sistema, abordando inicialmente os conceitos de hipertrofia muscular e as variáveis de controle do treinamento, como volume semanal, RPE e RIR. Em seguida, são discutidos os fundamentos tecnológicos que apoiarão o desenvolvimento da aplicação, como o padrão arquitetural MVC, a metodologia Desenvolvimento Guiado por Funcionalidades, do inglês *Feature-Driven Development* (FDD), React, Node.js, PostgreSQL, IA Generativa e engenharia de prompts. Por fim, são apresentados trabalhos correlatos que contribuem para a contextualização da pesquisa e para a identificação das limitações das soluções existentes.

A. Fisiologia da Hipertrofia e a Janela de Adaptação

O ganho de massa muscular, conhecido como hipertrofia, é o resultado das adaptações fisiológicas do corpo aos estímulos gerados pelo treinamento de força. O desenvolvimento muscular exige um estímulo contínuo que ultrapasse

o nível de adaptação atual da pessoa. A sinalização anabólica e o consequente acréscimo de massa muscular são mediados primariamente por três fatores interconectados: a tensão mecânica, o estresse metabólico e o dano muscular [2].

Dentre esses fatores, a tensão mecânica se destaca como o mais importante, ocorrendo quando as fibras musculares precisam lidar com uma carga excessiva. O estresse metabólico refere-se ao cansaço localizado e à 'queimação' gerada pelo esforço repetitivo do músculo. Por fim, o dano muscular envolve as pequenas lesões causadas pelo treinamento, que iniciam o processo de recuperação e crescimento do tecido [2]. Cabe ressaltar que a literatura científica evidencia que a tensão mecânica atua de forma sinérgica com o acúmulo de metabólitos. Dito isso, a combinação de alta tensão muscular com um estresse metabólico significativo maximiza o crescimento muscular, sendo consideravelmente mais eficiente para hipertrofia do que a tensão mecânica isolada [2]. Apesar dessa eficiência, a manutenção desse processo a longo prazo enfrenta limites biológicos. Segundo Kraemer e Ratamess [3] para romper essa barreira adaptativa é necessário manipular as variáveis de treinamento de forma estruturada, visto que manter o mesmo estímulo em praticantes já experientes acaba levando à estagnação dos resultados.

Compreender essa dinâmica é fundamental para o planejamento a longo prazo, uma vez que a resposta do corpo ao treinamento não é linear e se altera conforme a experiência do indivíduo. Nos estágios iniciais, praticantes respondem de forma altamente positiva a quase qualquer estímulo físico. Nessa fase, o corpo passa por rápidas adaptações iniciais, permitindo que rotinas básicas, mesmo sem grande rigor no controle de cargas, gerem resultados significativos. Por outro lado, à medida que o praticante transita para os níveis intermediário e avançado, o organismo torna-se cada vez mais eficiente na execução dos movimentos e mais resistente ao estresse provocado pelo exercício. Essa resistência se deve ao fato de que a taxa de crescimento muscular é naturalmente atenuada naqueles que já possuem experiência consolidada em treinamento de força [7]. Como o corpo se adapta para minimizar os danos estruturais e suportar cargas, torna-se progressivamente mais difícil aumentar massa muscular, o que estreita de forma significativa a janela de adaptação.[2]

A partir desse estágio de desenvolvimento muscular, a evolução deixa de acontecer de maneira espontânea. Para que o desenvolvimento continue e o praticante não estagne no chamado platô, torna-se indispensável um controle rigoroso, metrificado e planejado das variáveis de treinamento, manipulando o esforço de forma calculada para garantir resultados contínuos. Schoenfeld [7] reforça essa necessidade ao apontar que indivíduos bem treinados só alcançam novos ganhos hipertróficos quando um estímulo de sobrecarga inédito é aplicado meticulosamente ao longo do tempo. Assim, a manipulação precisa das métricas de esforço deixa de ser opcional e passar a ser o fator determinante para o

sucesso do programa.

B. Monitoramento de Variáveis de Esforço: Volume, RPE e RIR

O controle do esforço para praticantes que já ultrapassaram a fase iniciante deixa de ser apenas sobre o peso anotado no treino e passa a exigir um planejamento estruturado. Para que o sistema consiga processar e sugerir ajustes de carga, ele precisa fundamentar-se em variáveis que a literatura moderna aponta como métricas de controle de volume e intensidade.

A variável quantitativa central a ser gerenciada pelo sistema é o volume de treinamento. Historicamente calculado pela multiplicação bruta de séries, repetições e carga, abordagens contemporâneas sugerem métricas mais aplicáveis no dia a dia. Conforme evidenciado por Schoenfeld, Ogborn e Krieger [5], o número de séries válidas, denominadas na literatura de língua inglesa como *work sets*, corresponde às séries levadas próximas à falha mecânica e realizadas por grupamento muscular ao longo da semana, sendo uma medida fidedigna para quantificar o estímulo hipertrófico.

Observa-se uma relação de dose-resposta em que volumes maiores tendem a gerar mais resultados, porém essa progressão é limitada pela capacidade de recuperação do indivíduo. Estudos apontam que a realização de 10 ou mais séries semanais por grupamento muscular está associada a maiores ganhos hipertróficos, estabelecendo um limiar mínimo de referência a ser utilizado pelo sistema na análise do volume semanal [5].

Contudo, monitorar apenas o volume é insuficiente sem métricas que avaliem a intensidade do esforço em cada série. Para reduzir essa subjetividade, o projeto utiliza métodos psicofísicos validados para a musculação: as escalas de RPE e RIR. Originalmente desenvolvida para o esforço cardiovascular, a escala RPE foi ajustada para o treinamento de força por Zourdos et al. [4]. O diferencial desse modelo é que ele quantifica o esforço não apenas pela percepção de cansaço, mas pela estimativa real de quantas repetições a mais o praticante conseguiria realizar com a técnica adequada antes da falha concêntrica.

A viabilidade dessa autorregulação foi estruturada por Helms et al. [6], demonstrando que o uso dessas escalas permite ajustes dinâmicos nas sessões. Na prática, se o sistema registra um RPE 9 (equivalente a 1 RIR), indica que o estímulo foi otimizado; por outro lado, um RPE abaixo de 6 pode indicar que o praticante experiente não está gerando uma intensidade suficiente para promover adaptações. A adoção dessa autorregulação é crucial pois o desempenho humano flutua diariamente devido a variáveis biológicas, como qualidade do sono, nutrição e estresse [6]. Dessa forma, o sistema auxilia na adequação da intensidade à capacidade real do usuário no dia do treino, oferecendo uma alternativa dinâmica à rigidez dos métodos baseados apenas em percentuais de carga pré-fixados.

O grande desafio, que justifica a automação desse processo, reside na complexidade analítica de cruzar esses dados. Correlacionar o volume semanal acumulado [5] com a intensidade flutuante aferida pelo RIR/RPE [6] demanda um acompanhamento rigoroso que o praticante dificilmente sustenta manualmente por longos períodos. Ao traduzir essa base matemática e estrutural em algoritmos, o sistema atua como uma ferramenta de monitoramento analítico. Ele sugere momentos oportunos para progressão da carga ou a implementação de blocos de recuperação (*deload*), minimizando vieses subjetivos na gestão do treinamento.

C. Engenharia de Software e Arquitetura do Sistema

Para permitir a coleta e o processamento das métricas de esforço, a construção do sistema fundamenta-se em princípios estabelecidos da engenharia de software. Essa abordagem estruturada visa simplificar a manutenção do código ao longo do tempo e orientar o desenvolvimento da solução de acordo com a demanda do usuário final. Ao contrário dos modelos de processos tradicionais, as metodologias ágeis atuam como uma estratégia para lidar com a complexidade dos projetos modernos, priorizando a adaptabilidade da equipe e a entrega contínua de incrementos de software operacionais [8].

Nesse sentido, a condução do desenvolvimento segue os princípios da metodologia ágil FDD. Essa abordagem de processo se baseia na decomposição de problemas em funcionalidades, que são pequenas funções que agregam valor ao cliente, e enfatiza a colaboração entre membros da equipe para organização das etapas do trabalho [9]. No jargão desta metodologia, uma funcionalidade é definida como uma pequena etapa ou função de negócio que seja passível de implementação e teste em um intervalo curto de tempo, tipicamente em duas semanas ou menos [8]. O modelo FDD organiza o ciclo de vida do software em processos iterativos de modelagem, planejamento, projeto e construção por funcionalidade. Essa dinâmica estabelece um equilíbrio, evitando longas fases de planejamento inicial, e, de maneira complementar, mitiga o risco e o retrabalho causados pelo início precipitado da codificação sem um entendimento sólido do domínio [9]. Isso proporciona organização e controle ao processo ágil, permitindo que a equipe foque em entregas de software operantes e de alta qualidade em curtos períodos de tempo [8].

Para estruturar e sustentar essas funcionalidades, a aplicação fundamenta-se no padrão de arquitetura MVC. A adoção deste padrão busca aplicar a "separação de interesses" (*separation of concerns*), dividindo o sistema em três partes com responsabilidades muito bem delineadas, o que possibilita uma aplicação mais fácil de manter e estender ao longo de sua vida útil. O Modelo encapsula o gerenciamento dos dados e as regras de negócio; a Visão apresenta a interface gráfica ao usuário; e o Controlador processa as

requisições recebidas, orquestrando a comunicação entre a interface e os dados [10].

A adoção do padrão MVC reflete diretamente no ecossistema tecnológico escolhido para materializar o sistema. A camada de Visão é construída utilizando a biblioteca React, que permite a criação de interfaces de usuário baseadas na composição de pequenos componentes visuais independentes e reutilizáveis [11]. Em vez de manipular os elementos do navegador de forma imperativa e custosa, o React utiliza um DOM virtual (*Virtual Document Object Model*) [12]. Na prática, quando o praticante insere novas métricas de esforço no sistema, a biblioteca calcula as diferenças nesse modelo em memória e atualiza na tela apenas os fragmentos de interface que realmente sofreram modificação. Essa abordagem técnica evita o recarregamento completo da página a cada interação, permitindo a construção de uma interface dinâmica de maneira estruturada e menos propensa a erros de atualização visual [11].

O Controlador é estabelecido na plataforma Node.js, que se destaca por operar com arquitetura orientada a eventos e um modelo de entrada e saída não bloqueante (*non-blocking I/O*) [13, 14]. Nessa abordagem, a aplicação solicita uma operação demorada (como o acesso ao banco de dados) e continua sua execução imediatamente, sem bloquear o sistema. A tarefa é delegada em segundo plano e, ao ser concluída, o mecanismo interno Loop de Eventos (*Event Loop*) é encarregado de recuperar o resultado e despachar para a sua respectiva função de tratamento (*callback*) [13]. Na prática, essa característica viabiliza o processamento assíncrono, permitindo que a aplicação trate requisições simultâneas de cálculo de métricas de esforço físico de forma otimizada, sem que o servidor fique ocioso.

Por fim, a camada de Modelo é estruturada no sistema de gerenciamento de banco de dados relacional PostgreSQL [15]. A utilização deste sistema assegura a integridade referencial, a persistência e a estruturação segura do volume de dados gerado. O armazenamento confiável do histórico de treinamentos e da evolução contínua das métricas de RIR, RPE e volume semanal [16] é o alicerce fundamental para que o motor de IA, acoplado posteriormente, consiga consumir os dados e gerar diagnósticos precisos.

D. IA Generativa e Engenharia de Prompts

No contexto de cenários repletos de variáveis multidimensionais, como o cruzamento do histórico de RPE, carga e volume de múltiplos grupamentos musculares, os sistemas baseados em regras computacionais estáticas apresentam limitações analíticas. Conforme fundamentado pela literatura de Engenharia de Software, problemas dessa complexidade, que não são passíveis de computação convencional ou de análise direta, buscam na IA uma solução adequada [8]. Para mitigar essa limitação técnica, o sistema integra a IA Generativa na camada de controle. No escopo atual do projeto, utiliza-se a família de Modelos de Linguagem

Grande (do inglês, *Large Language Models* - LLMs) Gemini, disponibilizada via API. Esses modelos apresentam capacidades avançadas de raciocínio lógico a partir de instruções detalhadas [17], dispensando a necessidade de um treinamento local complexo na infraestrutura da aplicação para a geração de recomendações personalizadas.

Contudo, a simples transmissão de dados brutos para a API não é suficiente. A eficácia da análise e do cruzamento das métricas depende diretamente da técnica utilizada para estruturar a comunicação com o modelo, prática conhecida como Engenharia de *Prompts* (*Prompt Engineering*). Conforme descrito por White et al. [18], essa engenharia consiste na utilização de catálogos e padrões arquiteturais de comandos para aprimorar, guiar e refinar a interação com os LLMs. De forma análoga aos padrões de projeto da engenharia de software tradicional, os Padrões de *Prompts* (*Prompt Patterns*) documentam abordagens reutilizáveis para delimitar o contexto e o formato da resposta gerada.

Ao aplicar essa engenharia no desenvolvimento do sistema, dois padrões fundamentais se destacam para a parametrização do modelo ao domínio da hipertrofia. O primeiro é o Padrão de Personificação (*Persona Pattern*), que permite instruir a IA a assumir um papel analítico delimitado. Essa técnica direciona a ferramenta para atuar estritamente como um especialista em fisiologia do exercício, avaliando os dados de esforço sob uma perspectiva técnica e acadêmica, ignorando variáveis fora de escopo [18].

Em complemento, para viabilizar a integração técnica entre a IA e a arquitetura do sistema, se aplica o Padrão de Formatação (*Template Pattern*). Na prática, o *backend* da aplicação processa as variáveis de esforço brutas inseridas pelo usuário e as injeta em um *template* de *prompt* estruturado. Esse padrão impõe restrições sintáticas diretas ao modelo, exigindo que as recomendações geradas obedeam a um formato de dados previsível, como a notação JSON (*JavaScript Object Notation*) [18]. Essa abordagem permite converter a IA em um motor ativo de recomendação controlável, evitando respostas em textos livres ou extensos e permitindo que as prescrições de progressão sejam lidas pelo Node.js e exibidas com coerência na interface em React.

E. Trabalhos Correlatos

Nesta seção são discutidos trabalhos relacionados ao escopo deste projeto, contribuindo para o contexto do estudo que será realizado.

Schroter e Figueiredo [19] desenvolveram uma aplicação web para gestão de treinos de musculação voltada a *personal trainers*, utilizando a plataforma .NET com arquitetura MVC e integração com a API do Google Gemini. O sistema conta com o cadastro de alunos, registro de anamnese, gerenciamento de exercícios e geração automatizada de fichas de treino. A funcionalidade central de IA recebe os dados da anamnese e o catálogo de exercícios do profissional e retorna uma sugestão estruturada de divisão de treino em formato

JSON, que o *personal trainer* pode revisar e aprovar antes de atribuir ao aluno. A validação do sistema foi conduzida por meio da implementação funcional e da verificação dos objetivos específicos propostos.

A contribuição deste trabalho para o presente projeto está na demonstração prática da viabilidade técnica de integrar a API Gemini a uma aplicação MVC para geração de conteúdo estruturado em JSON a partir de um *template* de *prompt*. Contudo, o sistema proposto por Schroter e Figueiredo é orientado à gestão administrativa do profissional, não ao monitoramento contínuo das variáveis de esforço do praticante. Não há registro de RPE ou RIR por série, nem cálculo de volume semanal por grupamento muscular. O público-alvo também difere: enquanto aquele sistema apoia o *personal trainer* na prescrição, o presente trabalho destina-se ao próprio praticante experiente, que necessita de controle autônomo e rigoroso das métricas de intensidade descritas por Zourdos et al. [4] e Helms et al. [6] para evitar a estagnação apontada por Schoenfeld [2].

Noteboom et al. [20] desenvolveram e avaliaram uma aplicação de *feedback* de carga muscular integrada ao sistema de rastreamento automático da empresa Gymstory. O estudo distribuiu 30 participantes em três grupos, controle, *feedback* parcial e *feedback* completo, e monitorou oito sessões de treino. O grupo que recebeu o *feedback* completo, composto por um mapa corporal de carga muscular e sugestões de exercícios subsequentes, obteve uma distribuição de carga significativamente mais equilibrada entre os grupamentos musculares em comparação ao grupo controle ($\beta = -18,9$; IC 95% $[-29,3; -8,6]$). Adicionalmente, os autores verificaram que os grupos com *feedback* mantiveram o volume de carga mais estável ao longo das sessões, enquanto o grupo controle apresentou aumento progressivo, sugerindo risco de sobrecarga. A experiência do usuário foi avaliada positivamente no grupo de *feedback* completo para as dimensões de Atratividade, Estimulação e Novidade.

Os resultados de Noteboom et al. [20] são relevantes para o presente trabalho por duas razões. Primeiro, validam empiricamente que o *feedback* estruturado sobre variáveis de carga durante o treinamento de força produz resultados notáveis no comportamento do praticante, sustentando a ideia central deste projeto. Segundo, mostra uma limitação diretamente relacionada ao diferencial proposto, o modelo de estimativa de carga utilizado pelos autores se baseia em uma equação simplificada derivada de percentuais fixos de 1RM, com validade que os próprios autores reconhecem não ter sido investigada. Essa abordagem não captura a variação diária de desempenho, não considera a proximidade da falha muscular e não distingue séries válidas de séries não efetivadas. O presente trabalho supera essa limitação ao adotar as escalas de RPE e RIR como métricas de intensidade relativa [4, 6], que quantificam diretamente a proximidade da falha concêntrica em cada série válida registrada, independentemente de percentuais de carga fixos.

Essa diferença é especialmente relevante para praticantes experientes, que apresentam eficiência motora superior e cujo desempenho real flutua de forma significativa em razão de variáveis biológicas cotidianas [6]. Além disso, o presente sistema integra IA Generativa para processar o cruzamento longitudinal dessas métricas e gerar diagnósticos, funcionalidade ausente no trabalho de Noteboom et al.

A partir da análise dos trabalhos relacionados, observa-se que as soluções existentes concentram-se em dois eixos principais: a gestão administrativa de treinos e o feedback visual de carga muscular. O presente projeto diferencia-se ao integrar o registro de variáveis flutuantes e quantitativas de esforço, como RPE, RIR e volume semanal, com a análise ativa e geração de diagnósticos por IA generativa.

III. METODOLOGIA

Este capítulo descreve os procedimentos adotados para a modelagem e o desenvolvimento do protótipo proposto. A metodologia busca alinhar os conceitos da Ciência do Exercício às práticas de Engenharia de Software, de modo que as variáveis de esforço discutidas no referencial teórico sejam traduzidas em funcionalidades, estruturas de dados e fluxos de processamento. Para isso, serão utilizados artefatos de análise e projeto de software, como requisitos, diagramas de modelagem, arquitetura do sistema e definição dos procedimentos de validação [8].

A. Abordagem de Desenvolvimento

A condução do projeto fundamenta-se na metodologia ágil FDD. A escolha dessa abordagem justifica-se pela sua capacidade de lidar com a complexidade de domínios específicos e pela ênfase em resultados iterativos focados no cliente [9]. No contexto deste projeto, o FDD atua como a ponte estrutural para unir os requisitos biológicos da Ciência do Exercício com a arquitetura de *software*, decompondo o domínio do problema em pequenas entregas de valor controláveis e testáveis [8].

A primeira etapa, o Desenvolvimento de um Modelo Geral, exigiu a tradução direta das evidências científicas para o esquema lógico do sistema. A modelagem do banco de dados foi estruturada para suportar o rastreamento longitudinal da relação dose-resposta evidenciada por Schoenfeld, Ogborn e Krieger [5], por meio da contagem de séries semanais efetivas por grupamento muscular. Paralelamente, as tabelas precisaram contar com o monitoramento das métricas de RIR e RPE, validando as variações diárias de desempenho documentadas por Zourdos et al. [4].

Na segunda etapa, a Construção da Lista de Funcionalidades, os conceitos da fisiologia da hipertrofia foram convertidos para a sintaxe padrão do FDD [9], definindo o *backlog* do sistema:

- Realizar o cadastro e autenticação do usuário;
- Gerenciar a divisão e os exercícios da rotina semanal;

- Executar a sessão de treino com registro do tipo de séries (aquecimento ou válida), carga, repetições, RPE e RIR;
- Calcular o volume semanal de séries efetivas por grupamento muscular;
- Gerar o diagnóstico de treinamento via inteligência artificial;
- Visualizar o histórico de progressão de cargas e volume.

A tabela I detalha o cruzamento entre as cinco etapas iterativas do FDD [9, 8] e os artefatos técnicos gerados em cada fase do desenvolvimento.

Tabela I
APLICAÇÃO DAS ETAPAS DO FDD NO DESENVOLVIMENTO DO SISTEMA

Etapas do FDD	Foco metodológico	Entregável técnico
Modelo geral	Visão do domínio e estrutura do problema	DER com as entidades de usuário, treino, série, exercício, grupamento muscular e diagnóstico
Lista de funcionalidades	Decomposição do valor para o usuário	Backlog priorizado com funcionalidades de registro de esforço e diagnóstico via IA
Planejamento	Sequência e dependências entre entregas	Ordem de construção das camadas MVC e cronograma de desenvolvimento
Design	Detalhamento arquitetural por funcionalidade	Template de prompt e protótipos das interfaces gráficas em React
Construção	Implementação, testes e integração	Código-fonte em Node.js, integração com a API Gemini e testes de unidade e integração

Fonte: elaborado pelo autor.

Após a definição do backlog, a etapa de planejamento por funcionalidade estabelece a ordem de implementação dos módulos, considerando as dependências entre autenticação, cadastro de rotinas, registro de séries, cálculo de volume e geração de diagnósticos. Em seguida, o design por funcionalidade detalha a estrutura técnica de cada entrega, incluindo os fluxos de interface, regras de negócio, consultas ao banco de dados e templates de prompt. Por fim, a etapa de construção envolve a implementação incremental das funcionalidades, seguida por testes unitários e de integração.

B. Arquitetura e Modelagem do Sistema

A arquitetura do sistema foi definida a partir dos requisitos derivados do referencial teórico e dos objetivos do projeto. Como a proposta depende do registro estruturado das variáveis de esforço, do cálculo de indicadores de treino e da geração de diagnósticos por IA Generativa, a modelagem buscou representar as funcionalidades, os conceitos do domínio, os componentes da aplicação, o fluxo da funcionalidade principal e a estrutura de persistência dos dados.

Os requisitos funcionais delimitam as operações essenciais do protótipo, abrangendo cadastro, gerenciamento da

rotina de treino, registro de séries, cálculo de volume semanal e geração de diagnósticos pela API Gemini, conforme apresentado na Tabela II.

Tabela II
REQUISITOS FUNCIONAIS DO SISTEMA

ID	Descrição
RF01	Permitir cadastro e autenticação do usuário.
RF02	Permitir a criação e o gerenciamento da divisão semanal de treinos e exercícios.
RF03	Registrar séries executadas com tipo (aquecimento ou válida), carga, repetições, RPE e RIR.
RF04	Calcular automaticamente o volume semanal de séries efetivas por grupamento muscular.
RF05	Consolidar métricas semanais e enviá-las à API Gemini para geração de diagnóstico.
RF06	Persistir os diagnósticos gerados pela Inteligência Artificial.
RF07	Exibir o histórico de cargas registradas, volume semanal e diagnósticos anteriores.

Fonte: elaborado pelo autor.

Os requisitos não funcionais orientam aspectos de qualidade relacionados à usabilidade, integridade dos dados, organização das regras de negócio e controle da comunicação com a API externa, conforme apresentado na Tabela III.

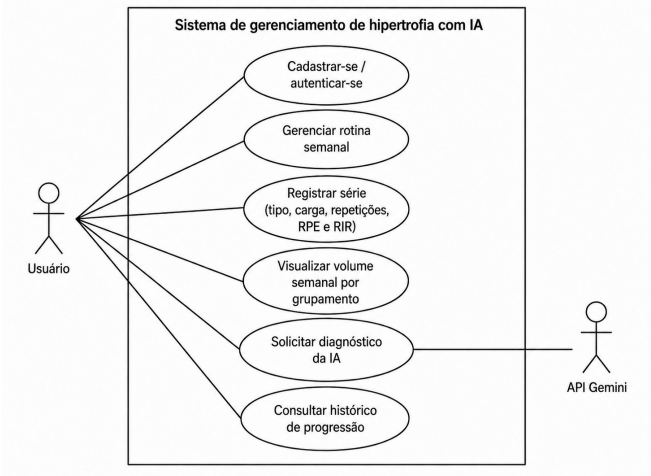
Tabela III
REQUISITOS NÃO FUNCIONAIS DO SISTEMA

ID	Descrição
RNF01	A interface deve ser responsiva e adequada ao uso em diversos dispositivos durante o treino.
RNF02	O sistema deve garantir integridade referencial no PostgreSQL.
RNF03	O backend deve centralizar os cálculos de volume, RPE e RIR.
RNF04	A comunicação com a API Gemini deve utilizar um <i>template de prompt</i> estruturado.
RNF05	O retorno da IA deve seguir formato estruturado, preferencialmente em JSON.
RNF06	O sistema deve preservar os registros de treino em caso de falha na API externa.

Fonte: elaborado pelo autor.

Com base nos requisitos, foi elaborado o Diagrama de Caso de Uso, que representa as interações externas do usuário com as funcionalidades principais do sistema. A API Gemini aparece como ator secundário por participar apenas da geração do diagnóstico de treino.

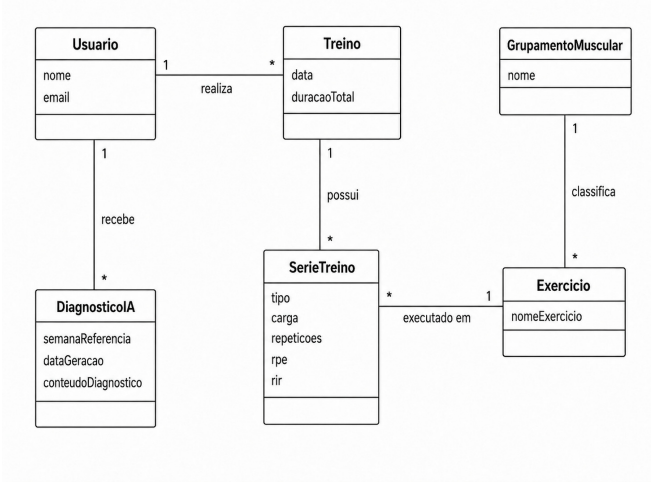
Figura 1. Diagrama de Caso de Uso do sistema



Fonte: Elaborado pelo autor

O Diagrama de Domínio apresenta os principais conceitos envolvidos no problema, evidenciando as relações entre usuário, treino, séries executadas, exercícios, grupamentos musculares e diagnósticos gerados pela IA.

Figura 2. Diagrama de Domínio do sistema



Fonte: Elaborado pelo autor

A partir da modelagem do domínio, os conceitos centrais do referencial teórico foram traduzidos em regras computacionais. Essa operacionalização permite relacionar as variáveis discutidas na literatura com as funcionalidades implementadas no sistema, conforme apresentado na Tabela IV.

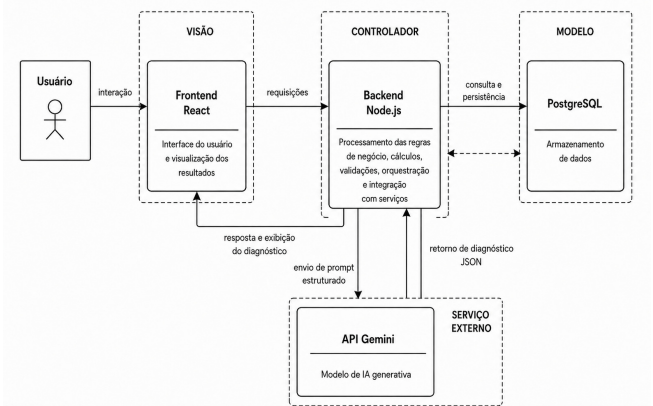
Tabela IV
CONCEITOS TEÓRICOS TRADUZIDOS EM LÓGICA DE IMPLEMENTAÇÃO

Referência	Conceito teórico	Lógica de implementação
Schoenfeld et al. [5]	Volume de treino	Soma semanal de séries válidas por grupamento muscular e comparação com limiar mínimo de referência.
Zourdos et al. [4]	Intensidade relativa	Registro de RPE e RIR por série válida para estimar a proximidade da falha muscular.
Helms et al. [6]	Autorregulação	Interpretação da carga com base no desempenho registrado e no esforço percebido pelo usuário.

Fonte: Elaborado pelo autor

A arquitetura do protótipo foi organizada segundo o padrão MVC [10]. A interface em React compõe a camada de visão; o *backend* em Node.js atua como controlador, concentrando regras de negócio, cálculos e integração com serviços externos; e o PostgreSQL representa a camada de modelo, responsável pela persistência dos dados. A API Gemini atua como serviço externo consumido pelo *backend*.

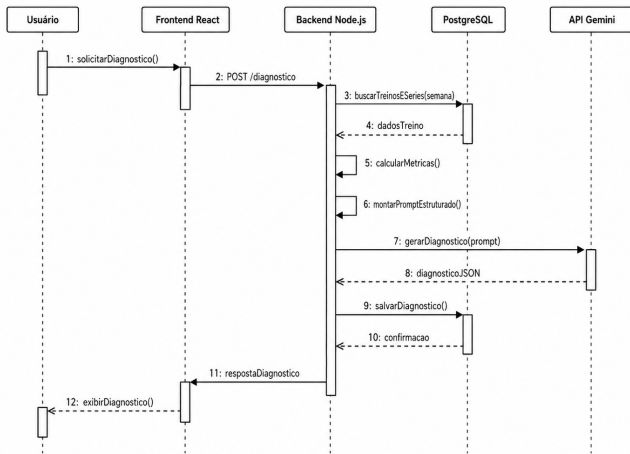
Figura 3. Diagrama de Componentes da arquitetura proposta



Fonte: Elaborado pelo autor

A geração do diagnóstico foi detalhada por meio do Diagrama de Sequência, que descreve a comunicação entre usuário, interface React, *backend* Node.js, banco de dados PostgreSQL e API Gemini, desde a solicitação do diagnóstico até a persistência e exibição.

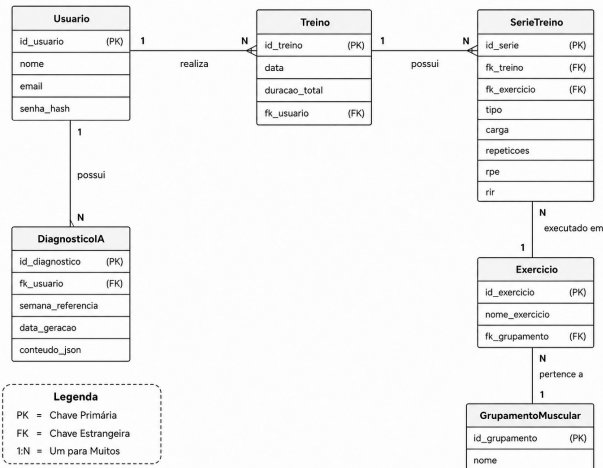
Figura 4. Diagrama de Sequência da geração de diagnóstico via IA



Fonte: Elaborado pelo autor

Por fim, a persistência dos dados foi representada pelo Diagrama Entidade-Relacionamento. A modelagem contempla as entidades necessárias para armazenar usuários, treinos, exercícios, séries executadas, grupamentos musculares e diagnósticos. A entidade *SerieTreino* concentra tipo da série, carga, repetições, RPE e RIR, que compõem as variáveis centrais de análise do sistema. Já a entidade *Treino* armazena informações de organização da sessão, como data e duração total, utilizadas para exibição histórica e acompanhamento do usuário. A entidade *Exercicio* é associada a um grupamento muscular principal, utilizado como referência para a consolidação do volume semanal. A entidade *DiagnosticoIA* armazena os pareceres gerados pela API Gemini.

Figura 5. Diagrama Entidade-Relacionamento do sistema



Fonte: Elaborado pelo autor

C. Integração com Inteligência Artificial Generativa

A integração com o Google Gemini será realizada via API, aplicando técnicas de Engenharia de *Prompt* para estruturar a comunicação entre o *backend* e o modelo de linguagem [18]. O sistema utilizará o *Template Pattern* [18], em que o *prompt* enviado à IA não é uma entrada livre do usuário, mas uma estrutura predefinida que combina cinco blocos funcionais: a *persona* analítica, o contexto da semana avaliada, os dados brutos de treino extraídos do PostgreSQL, as diretrizes científicas e as diretrizes de formato de saída.

O *Persona Pattern* [18] instruirá o modelo a assumir o papel de especialista em fisiologia do exercício, restringindo o escopo das recomendações ao domínio do treinamento de força com foco em hipertrofia. Essa delimitação permite reduzir o risco de respostas fora de contexto e de alucinações do modelo [17]. O bloco de dados injetará as métricas consolidadas da semana (volume de séries válidas por grupamento muscular, RPE e RIR registrados por série) diretamente no *template*, convertendo os registros do banco em insumos analíticos estruturados. O bloco de diretrizes científicas possibilita delimitar os critérios de avaliação com base no referencial teórico adotado: o limiar mínimo de séries semanais por grupamento muscular fundamentado em Schoenfeld, Ogborn e Krieger [5] e os parâmetros de intensidade relativa definidos por Zourdos et al. [4] e Helms et al. [6].

A Tabela 5 apresenta a anatomia do *prompt* estruturado a ser utilizado pelo sistema, detalhando cada bloco, seu conteúdo e o padrão de *prompt* que será aplicado.

Tabela V
ANATOMIA DO PROMPT ESTRUTURADO

Bloco	Conteúdo injetado	Padrão
<i>Persona</i>	Instrução que define o papel analítico do modelo: especialista em fisiologia do exercício, avaliando dados sob perspectiva técnica e restrita ao domínio do treinamento de força.	<i>Persona Pattern</i> [18]
Contexto	Semana de referência e identificação do conjunto de grupamentos musculares treinados no período.	<i>Template Pattern</i> [18]
Dados de treino	Volume semanal de séries válidas por grupamento muscular, RPE e RIR registrados por série, extraídos do PostgreSQL.	<i>Template Pattern</i> [18]
Diretrizes científicas	Limiar mínimo de volume semanal [5]; parâmetros de RPE e RIR para intensidade adequada [4, 6].	<i>Template Pattern</i> [18]
Formato de saída	Instrução que exige retorno exclusivamente em objeto JSON com campos predefinidos, sem texto livre adicional.	<i>Template Pattern</i> [18]

Fonte: Elaborado pelo autor

O retorno da API Gemini deve seguir formato JSON estruturado, contendo campos analíticos como `diagnostico_exercicios`, `analise_grupamentos` e `recomendacoes_proxima_sessao`. Após o recebimento desse retorno, o *backend* em Node.js calculará o campo `score_geral`, aplicando regras determinísticas baseadas nos limiares definidos no referencial teórico, e então persistirá o diagnóstico consolidado para exibição na interface em React.

D. Procedimentos de Validação

A validação do protótipo é organizada em três frentes complementares, embasadas em práticas consolidadas da engenharia de software [8].

A primeira frente consiste na realização de testes de unidade e de integração. Os testes de unidade verificarão isoladamente a corretude dos cálculos de volume semanal por grupamento muscular (RF04), comparando os resultados produzidos pelo *backend* com valores calculados manualmente a partir de cenários de treino conhecidos. Os testes de integração verificarão a comunicação entre os módulos do sistema, assegurando que o fluxo completo de envio de métricas à API Gemini (RF05), a persistência do diagnóstico retornado (RF06) e a exibição do histórico ao usuário (RF07) operem sem falhas de execução. Essa frente cobrirá diretamente os requisitos RNF02, RNF03 e RNF06.

A segunda frente consiste na avaliação de usabilidade da interface gráfica, a ser conduzida por meio de inspeção analítica baseada nas heurísticas de Nielsen [21]. A avaliação focará nos fluxos de uso mais críticos do sistema: o registro de séries com RPE e RIR durante a sessão de treino (RF03) e a visualização do diagnóstico gerado pela IA (RF05 e RF07). O critério de aceitação para essa frente será a ausência de violações graves de usabilidade nos fluxos avaliados, especialmente em dispositivos diversos (RNF01).

A terceira frente consiste na validação de conteúdo dos diagnósticos gerados pela IA. Cenários de treino simulados, com métricas de volume, RPE e RIR previamente conhecidas, serão submetidos ao sistema para verificar se os diagnósticos retornados são coerentes com o que a literatura indica para aquela combinação de variáveis [5, 4, 6]. Essa frente validará a eficácia do *template* de *prompt* estruturado (RNF04) em restringir as inferências do modelo ao domínio fisiológico esperado.

A Tabela 6 apresenta a matriz de rastreabilidade, correlacionando cada requisito ao seu respectivo método de validação e critério de aceite.

Tabela VI
MATRIZ DE RASTREABILIDADE DOS REQUISITOS

ID	Método de validação	Critério de aceite
RF01	Teste de unidade	Autenticação sem erros
RF02	Teste de unidade e usabilidade	Operações CRUD sem falhas
RF03	Teste de unidade e usabilidade	Campos persistidos corretamente
RF04	Teste de unidade	Resultado igual ao cálculo manual
RF05	Teste de integração e validação de conteúdo	Diagnóstico retornado em JSON válido
RF06	Teste de integração	Diagnóstico salvo no banco após retorno
RF07	Teste de integração e usabilidade	Dados exibidos corretamente na interface
RNF01	Avaliação de usabilidade	Sem violações graves nos fluxos principais
RNF02	Teste de integração	Rejeição de registros órfãos
RNF03	Teste de unidade	Todos os cálculos de volume, RPE e RIR executados exclusivamente no <i>backend</i>
RNF04	Validação de conteúdo	Diagnóstico dentro do domínio fisiológico
RNF05	Teste de integração	JSON parseável sem erros pelo Node.js
RNF06	Teste de integração	Registros intactos após falha simulada

Fonte: Elaborado pelo autor

IV. CONCLUSÕES PARCIAIS

Este projeto apresentou a fundamentação teórica e o planejamento para o desenvolvimento de um sistema web integrado com IA Generativa, focado no gerenciamento do treinamento de hipertrofia para praticantes experientes. Inicialmente, foram discutidos os mecanismos fisiológicos do ganho de massa muscular e a necessidade de um controle rigoroso das variáveis de esforço. Em seguida, definiu-se a

base tecnológica da solução, englobando o padrão arquitetural MVC (composto por React, Node.js e PostgreSQL) e as técnicas de Engenharia de Prompts necessárias para a integração com a API do Google Gemini.

Em relação aos resultados obtidos nesta primeira etapa, destaca-se a modelagem estrutural do sistema. Essa fase foi documentada por meio dos diagramas de Caso de Uso, Domínio, Componentes, Sequência e Entidade-Relacionamento. Na prática, esses modelos serviram para transformar as regras fisiológicas mapeadas na literatura em requisitos de software. Além disso, a aplicação da metodologia FDD permitiu a construção de um *backlog* de funcionalidades priorizado, assim como a definição da estrutura do prompt que será enviado ao modelo Gemini, concluindo a etapa de projeto e design do protótipo.

Para a continuidade do trabalho, que corresponde à etapa de Trabalho de Conclusão de Curso II (TCC II), o foco será a implementação do código-fonte do sistema. Isso inclui a programação da camada de visão em React, a construção do controlador em Node.js, a configuração do banco de dados em PostgreSQL e a efetivação da comunicação com a API Gemini. Após a construção do protótipo, o sistema passará pelos procedimentos de validação definidos na metodologia: testes de unidade e de integração, avaliação heurística de usabilidade e validação de conteúdo dos diagnósticos gerados pela IA. O objetivo dessa etapa final será verificar se o sistema entrega respostas coerentes com a literatura científica e se a interface atende de forma eficiente o usuário durante o treino.

Por fim, a distribuição temporal de todas essas atividades está organizada no cronograma da Figura 6. O primeiro semestre concentrou-se na revisão da literatura bibliográfica, no levantamento de requisitos, na modelagem do sistema e na escrita do documento de TCC I. O segundo semestre, por sua vez, será dedicado à fase de desenvolvimento, o que compreende a codificação, a integração com a IA, a execução dos testes de validação e a redação e defesa final do TCC II.

Figura 6. Cronograma do trabalho de TCC.

Atividades	Mar	Abr	Mai	Jun	Jul	Ago	Set	Out	Nov
Escrita do Pré-Projeto	X								
Revisão Bibliográfica (Hipertrofia e Esforço)	X	X							
Levantamento de Requisitos e Modelagem FDD		X	X						
Definição da Arquitetura MVC e Banco de Dados			X						
Escrita e Entrega do TCC 1		X	X	X					
Desenvolvimento da Interface e servidor						X	X	X	
Implementação de Banco de Dados							X	X	
Integração com API Gemini								X	X
Testes de Validação do Diagnóstico de Hipertrofia									X
Escrita e Defesa do TCC 2						X	X	X	X

Fonte: Elaborado pelo autor

REFERÊNCIAS

- [1] Paulina Bondaronek et al. “Quality of Publicly Available Physical Activity Apps: Review and Content Analysis”. Em: *JMIR mHealth and uHealth* 6.3 (2018), e53.
- [2] Brad J Schoenfeld. “The mechanisms of muscle hypertrophy and their application to resistance training”. Em: *The Journal of Strength & Conditioning Research* 24.10 (2010), pp. 2857–2872.
- [3] William J Kraemer e Nicholas A Ratamess. “Fundamentals of resistance training: progression and exercise prescription”. Em: *Medicine & Science in Sports & Exercise* 36.4 (2004), pp. 674–688.
- [4] Michael C Zourdos et al. “Novel resistance training-specific rating of perceived exertion scale measuring repetitions in reserve”. Em: *The Journal of Strength & Conditioning Research* 30.1 (2016), pp. 267–275.
- [5] Brad J Schoenfeld, Dan Ogborn e James W Krieger. “Dose-response relationship between weekly resistance training volume and increases in muscle mass: A systematic review and meta-analysis”. Em: *Journal of Sports Sciences* 35.11 (2017), pp. 1073–1082.
- [6] Eric R Helms et al. “Application of the repetitions in reserve-based rating of perceived exertion scale for resistance training”. Em: *Strength & Conditioning Journal* 38.4 (2016), pp. 42–49.
- [7] Brad J Schoenfeld, Dan Ogborn e James W Krieger. “Effects of resistance training frequency on hypertrophic outcomes: a systematic review and meta-analysis”. Em: *Sports Medicine* 46.11 (2016), pp. 1689–1697.
- [8] Roger S Pressman. *Engenharia de Software: uma abordagem profissional*. 7ª ed. Porto Alegre: AMGH Editora, 2011.
- [9] Steve R Palmer e Mac Felsing. *A practical guide to feature-driven development*. Pearson Education, 2001.
- [10] Adam Freeman. *Pro ASP.NET Core MVC*. 6ª ed. Berkeley, CA: Apress, 2018.
- [11] Alex Banks e Eve Porcello. *Learning React: Modern Patterns for Developing React Apps*. 2ª ed. Sebastopol: O’Reilly Media, 2020.
- [12] React. *React Documentation*. <https://react.dev/>. Acesso em: 10 mar. 2026. 2026.
- [13] Mario Casciaro e Luciano Mammino. *Node.js Design Patterns*. 3ª ed. Birmingham: Packt Publishing, 2020.
- [14] Node.js. *Node.js v20.x documentation*. <https://nodejs.org/docs/latest-v20.x/api/>. Acesso em: 10 mar. 2026. 2026.
- [15] The PostgreSQL Global Development Group. *PostgreSQL: Documentation*. <https://www.postgresql.org/docs/>. Acesso em: 10 mar. 2026. 2026.

- [16] Abraham Silberschatz, Henry F Korth e S Sudarshan. *Database System Concepts*. 7ª ed. New York: McGraw-Hill, 2019.
- [17] Gemini Team et al. “Gemini: a family of highly capable multimodal models”. Em: *arXiv preprint arXiv:2312.11805* (2023).
- [18] Jules White et al. “A prompt pattern catalog to enhance prompt engineering with ChatGPT”. Em: *arXiv preprint arXiv:2302.11382* (2023).
- [19] Leonardo Barros Schroter e Herysson Rodrigues Figueiredo. *Sistema De Gestão De Treinos De Musculação*. Santa Maria, RS, Brasil. Disponível em <https://tfgonline.lapinf.ufn.edu.br>: Trabalho de Conclusão de Curso Sistemas De Informação - Universidade Franciscana (UFN), 2025.
- [20] Lisa Noteboom et al. “A muscle load feedback application for strength training: A proof-of-concept study”. Em: *Sports* 11.9 (2023), p. 170.
- [21] Jakob Nielsen. “Enhancing the explanatory power of usability heuristics”. Em: *Proceedings of the SIGCHI conference on Human Factors in Computing Systems*. 1994, pp. 152–158.